

Generative Inference

COMP4551

Binhang Yuan

Recall Language Modeling

What Is a Language Model?

- The classic definition of a *language model (LM)* is a probability distribution over sequences of tokens.
- Suppose we have a vocabulary $\mathcal{V}$ of a set of tokens.
- A language model P assigns each sequence of tokens $x_1, x_2, \dots, x_L \in \mathcal{V}$ to a probability (a number between 0 and 1): $p(x_1, x_2, \dots, x_L) \in [0,1]$.
- The probability intuitively tells us how “good” a sequence of tokens is.
 - For example, if the vocabulary is $\mathcal{V} = \{\text{ate, ball, cheese, mouse, the}\}$, the language model might assign:
$$p(\text{the, mouse, ate, the, cheese}) = 0.02$$
$$p(\text{the, cheese, ate, the, mouse}) = 0.01$$
$$p(\text{mouse, the, the, chesse, ate}) = 0.0001$$

Decoder-only Models

- Decoder-only models are our standard autoregressive language models.
- Given a prompt $x_{1:i}$ produces both contextual embeddings and a distribution over next tokens x_{i+1} , and recursively, over the entire completion $x_{i+1:L}$:

$$x_{1:i} \Rightarrow \phi(x_{1:i}), p(x_{i+1}|x_{1:i})$$

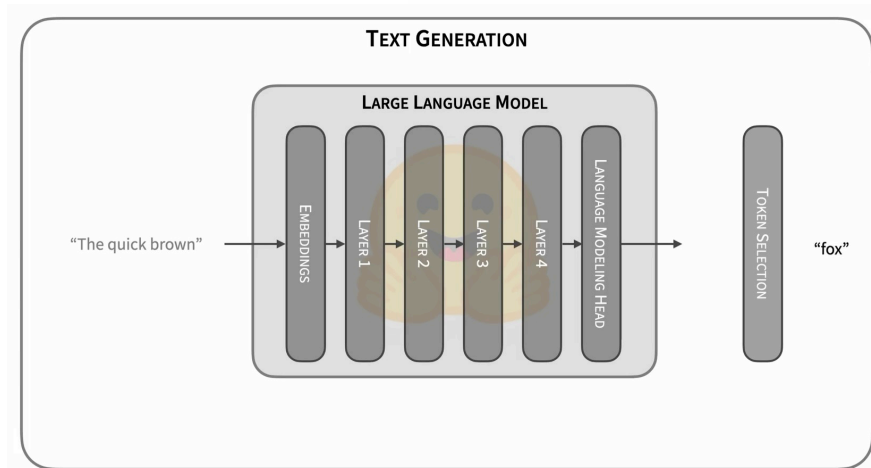
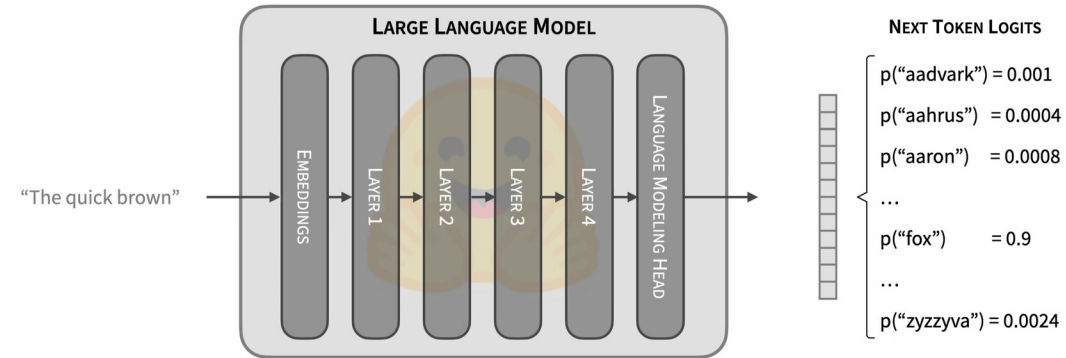
- Example: text autocomplete
 - $[[CLS], \text{the, movie, was}] \Rightarrow \text{great}$
- The probability $p(x_{i+1}|x_{1:i})$ is usually determined by:
$$p(x_{i+1}|x_{1:i}) = \text{softmax}(X_i W_{lm}), X_i \in \mathbb{R}^D, W_{lm} \in \mathbb{R}^{D \times |\mathcal{V}|}$$

Autoregressive Generation

Autoregressive Generation

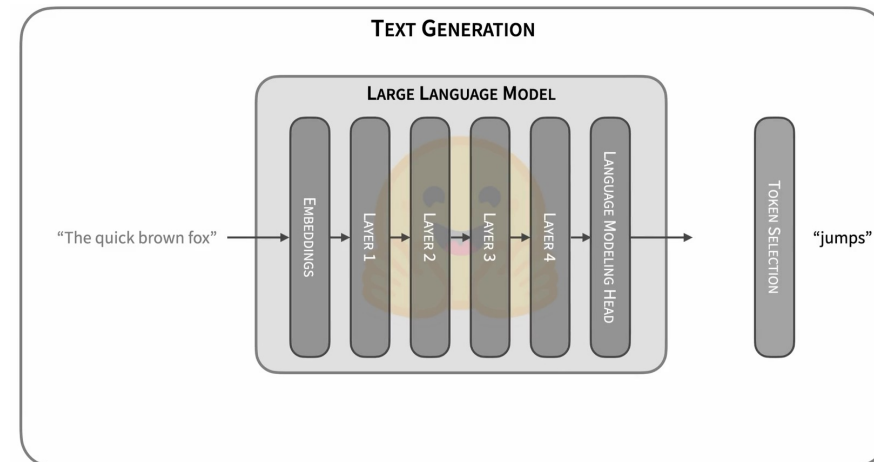
- LLM takes a sequence of text tokens as input and returns the probability distribution for the next token.
- A critical aspect of autoregressive generation with LLMs is how to select the next token from this probability distribution.
- There are many ways in this step as long as you end up with a token for the next iteration.
- The simplest way is to **select the most likely token from the probability distribution**.
- More complex solutions, e.g., applying a dozen transformations before sampling from the resulting distribution.

Autoregressive Generation



The quick brown => fox

Step 1



The quick brown fox => jumps

Step 2

Naïve Implementation

TransformerBlocks($X \in \mathbb{R}^{L \times D}$) $\rightarrow X' \in \mathbb{R}^{L \times D}$

For each inference request:

- ~~$B = 1$~~ ;
- L is the input sequence length;
- D is the model dimension;
- Multi-head attention:
 $D = n_H \times H$
- H is the head dimension;
- n_h is the number of heads.

Computation	Input	Output
$Q = XW^Q$	$X \in \mathbb{R}^{L \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q \in \mathbb{R}^{L \times D}$
$K = XW^K$	$X \in \mathbb{R}^{L \times D}, W^K \in \mathbb{R}^{D \times D}$	$K \in \mathbb{R}^{L \times D}$
$V = XW^V$	$X \in \mathbb{R}^{L \times D}, W^V \in \mathbb{R}^{D \times D}$	$V \in \mathbb{R}^{L \times D}$
$[Q_1, Q_2 \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{L \times D}$	$Q_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$[K_1, K_2 \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{L \times D}$	$K_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$[V_1, V_2 \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{L \times D}$	$V_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots n_h$	$Q_i, K_i \in \mathbb{R}^{L \times H}$	$\text{score}_i \in \mathbb{R}^{L \times L}$
$Z_i = \text{score}_i V_i, i = 1, \dots n_h$	$\text{score}_i \in \mathbb{R}^{L \times L}, V_i \in \mathbb{R}^{L \times H}$	$Z_i \in \mathbb{R}^{L \times H}$
$Z = \text{Merge}_{-1}([Z_1, Z_2 \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$	$Z \in \mathbb{R}^{L \times D}$
$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{L \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{L \times D}$
$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{L \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{L \times 4D}$
$A' = \text{relu}(A)$	$A \in \mathbb{R}^{L \times 4D}$	$A' \in \mathbb{R}^{L \times 4D}$
$X' = A'W^2$	$A' \in \mathbb{R}^{L \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$X' \in \mathbb{R}^{L \times D}$

Generate the first token.

$$p(x_{L+1}|x_{1:L}) = \text{softmax}(X_L^{-1} W_{lm})$$

-1 indicates the last TransformerBlock



TransformerBlocks($X \in \mathbb{R}^{(L+1) \times D}$) $\rightarrow X' \in \mathbb{R}^{(L+1) \times D}$

For each inference request:

- $B = 1$;
- $L+1$ is the current input sequence length;
- D is the model dimension;
- Multi-head attention:
 $D = n_H \times H$
- H is the head dimension;
- n_h is the number of heads.

Computation	Input	Output
$Q = XW^Q$	$X \in \mathbb{R}^{(L+1) \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q \in \mathbb{R}^{(L+1) \times D}$
$K = XW^K$	$X \in \mathbb{R}^{(L+1) \times D}, W^K \in \mathbb{R}^{D \times D}$	$K \in \mathbb{R}^{(L+1) \times D}$
$V = XW^V$	$X \in \mathbb{R}^{(L+1) \times D}, W^V \in \mathbb{R}^{D \times D}$	$V \in \mathbb{R}^{(L+1) \times D}$
$[Q_1, Q_2, \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{(L+1) \times D}$	$Q_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$
$[K_1, K_2, \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{(L+1) \times D}$	$K_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$
$[V_1, V_2, \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{(L+1) \times D}$	$V_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$
$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots, n_h$	$Q_i, K_i \in \mathbb{R}^{(L+1) \times H}$	$\text{score}_i \in \mathbb{R}^{(L+1) \times (L+1)}$
$Z_i = \text{score}_i V_i, i = 1, \dots, n_h$	$\text{score}_i \in \mathbb{R}^{(L+1) \times (L+1)}, V_i \in \mathbb{R}^{(L+1) \times H}$	$Z_i \in \mathbb{R}^{(L+1) \times H}$
$Z = \text{Merge}_{-1}([Z_1, Z_2, \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$	$Z \in \mathbb{R}^{(L+1) \times D}$
$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{(L+1) \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{(L+1) \times D}$
$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{(L+1) \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{(L+1) \times 4D}$
$A' = \text{relu}(A)$	$A \in \mathbb{R}^{(L+1) \times 4D}$	$A' \in \mathbb{R}^{(L+1) \times 4D}$
$X' = A'W^2$	$A' \in \mathbb{R}^{(L+1) \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$X' \in \mathbb{R}^{(L+1) \times D}$

Generate the second token.

$$p(x_{L+2}|x_{1:L+1}) = \text{softmax}(X_{L+1}^{-1} W_{lm})$$

-1 indicates the last TransformerBlock

Reuse KV Cache

Some Observation

- Only the last contextual embedding is needed to compute the probabilistic distribution of the next token.
- Contextual embedding for the current token can only depend **unidirectionally** on the left context. In the previous naïve implementation, most of the computation is redundant.
- State-of-the-art implementation splits the computation to two phrases:
 - Prefill phrase: the model takes a prompt sequence as input and engages in the generation of a key-value cache (KV cache) for each Transformer layer.
 - Decode phrase: for each decode step, the model updates the KV cache and reuses the KV to compute the output.

Prefill: TransformerBlocks($X \in \mathbb{R}^{L \times D}$) $\rightarrow X' \in \mathbb{R}^{L \times D}$

For each inference request:

- ~~$B = 1$~~ ;
- L is the input sequence length;
- D is the model dimension;
- Multi-head attention:
 $D = n_H \times H$
- H is the head dimension;
- n_h is the number of heads.

Computation	Input	Output
$Q = XW^Q$	$X \in \mathbb{R}^{L \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q \in \mathbb{R}^{L \times D}$
$K = XW^K$	$X \in \mathbb{R}^{L \times D}, W^K \in \mathbb{R}^{D \times D}$	$K \in \mathbb{R}^{L \times D}$
$V = XW^V$	$X \in \mathbb{R}^{L \times D}, W^V \in \mathbb{R}^{D \times D}$	$V \in \mathbb{R}^{L \times D}$
$[Q_1, Q_2 \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{L \times D}$	$Q_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$[K_1, K_2 \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{L \times D}$	$K_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$[V_1, V_2 \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{L \times D}$	$V_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots n_h$	$Q_i, K_i \in \mathbb{R}^{L \times H}$	$\text{score}_i \in \mathbb{R}^{L \times L}$
$Z_i = \text{score}_i V_i, i = 1, \dots n_h$	$\text{score}_i \in \mathbb{R}^{L \times L}, V_i \in \mathbb{R}^{L \times H}$	$Z_i \in \mathbb{R}^{L \times H}$
$Z = \text{Merge}_{-1}([Z_1, Z_2 \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$	$Z \in \mathbb{R}^{L \times D}$
$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{L \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{L \times D}$
$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{L \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{L \times 4D}$
$A' = \text{relu}(A)$	$A \in \mathbb{R}^{L \times 4D}$	$A' \in \mathbb{R}^{L \times 4D}$
$X' = A'W^2$	$A' \in \mathbb{R}^{L \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$X' \in \mathbb{R}^{L \times D}$

Generate the first token.

$$p(x_{L+1}|x_{1:L}) = \text{softmax}(X_L^{-1} W_{lm})$$

-1 indicates the last TransformerBlock



Decode: TransformerBlocks($t \in \mathbb{R}^{1 \times D}$) $\rightarrow t' \in \mathbb{R}^{1 \times D}$

For each inference request:

- $B = 1$;
- L is the current cached sequence length; it increases by 1 after each step.
- D is the model dimension;
- Multi-head attention:
 $D = n_H \times H$
- H is the head dimension;
- n_h is the number of heads.

Update the KV cache:

$$K = \text{concat}(K_{\text{cache}}, K_d)$$

$$V = \text{concat}(V_{\text{cache}}, V_d)$$

Generate the second token:

$$p(x_{L+2}|x_{1:L+1}) = \text{softmax}(T_{L+1}^{-1} W_{lm})$$

-1 indicates the last TransformerBlock

Computation	Input	Output
$Q = Q_d = TW^Q$	$T \in \mathbb{R}^{1 \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q, Q_d \in \mathbb{R}^{1 \times D}$
$K_d = TW^K$	$T \in \mathbb{R}^{1 \times D}, W^K \in \mathbb{R}^{D \times D}$	$K_d \in \mathbb{R}^{1 \times D}$
$K = \text{concat}(K_{\text{cache}}, K_d)$	$K_{\text{cache}} \in \mathbb{R}^{L \times D}, K_d \in \mathbb{R}^{1 \times D}$	$K \in \mathbb{R}^{(L+1) \times D}$
$V_d = tW^V$	$t \in \mathbb{R}^{1 \times D}, W^V \in \mathbb{R}^{D \times D}$	$V_d \in \mathbb{R}^{1 \times D}$
$V = \text{concat}(V_{\text{cache}}, V_d)$	$V_{\text{cache}} \in \mathbb{R}^{L \times D}, V_d \in \mathbb{R}^{1 \times D}$	$V \in \mathbb{R}^{(L+1) \times D}$
$[Q_1, Q_2 \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{1 \times D}$	$Q_i \in \mathbb{R}^{1 \times H}, i = 1, \dots, n_h$
$[K_1, K_2 \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{(L+1) \times D}$	$K_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$
$[V_1, V_2 \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{(L+1) \times D}$	$V_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$
$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots, n_h$	$Q_i \in \mathbb{R}^{1 \times H}, K_i \in \mathbb{R}^{(L+1) \times H}$	$\text{score}_i \in \mathbb{R}^{1 \times (L+1)}$
$Z_i = \text{score}_i V_i, i = 1, \dots, n_h$	$\text{score}_i \in \mathbb{R}^{1 \times (L+1)}, V_i \in \mathbb{R}^{(L+1) \times H}$	$Z_i \in \mathbb{R}^{1 \times H}$
$Z = \text{Merge}_{-1}([Z_1, Z_2 \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{1 \times H}, i = 1, \dots, n_h$	$Z \in \mathbb{R}^{1 \times D}$
$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{1 \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{1 \times D}$
$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{1 \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{1 \times 4D}$
$A' = \text{relu}(A)$	$A \in \mathbb{R}^{1 \times 4D}$	$A' \in \mathbb{R}^{1 \times 4D}$
$T' = A'W^2$	$A' \in \mathbb{R}^{1 \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$T' \in \mathbb{R}^{1 \times D}$

Reuse the KV Cache

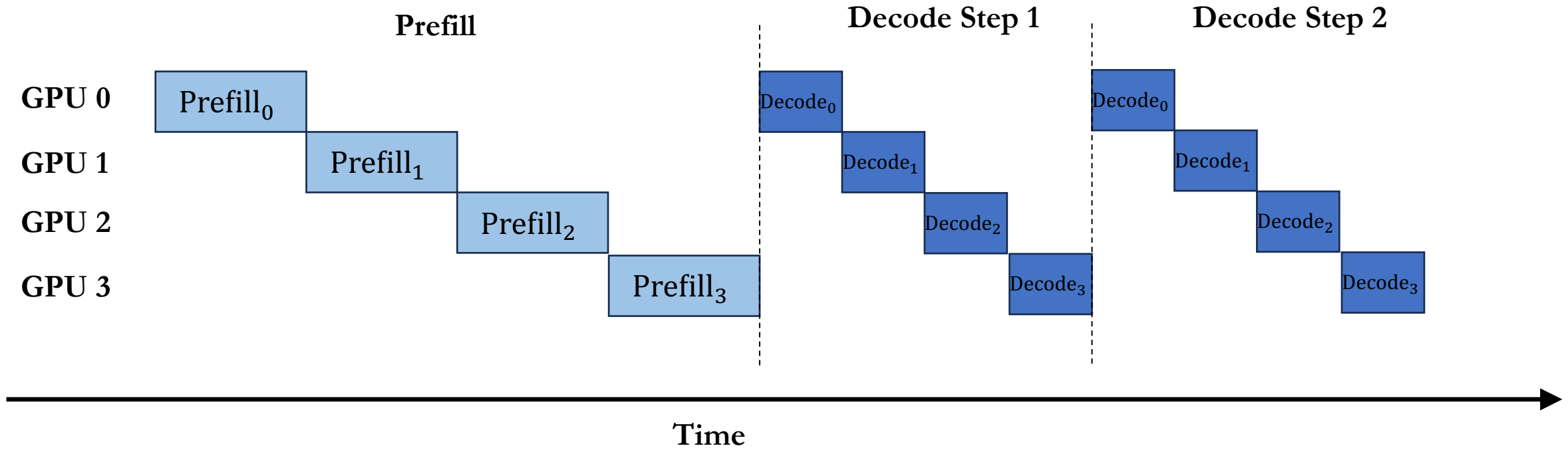
- Performance analysis of this computation paradigm:
 - Prefill phrase: computation bounded.
 - Decode phrase: IO bounded.
 - What is the arithmetic intensity of each step?
- The memory footprint of the generative inference computation:
 - Model parameters;
 - KV-cache. This will become more significant since the latest models are targeting long context comprehension.

Parallel Generative Inference

Pipeline Parallelism

- Similar to training, pipeline parallelism partitions the model into multiple stages and serves the inference computation as a pipeline, where each GPU or (group of GPUs) handles a stage.
- During the inference computation, the GPU(s) serving stage- (i) needs to **send** the activations to the GPU(s) serving stage- $(i + 1)$.
- For inference computation, pipeline parallelism **cannot** reduce the completion time for a single request since only one stage can be active.

Pipeline Parallelism for Inference.



The number on each block represents the stage index.

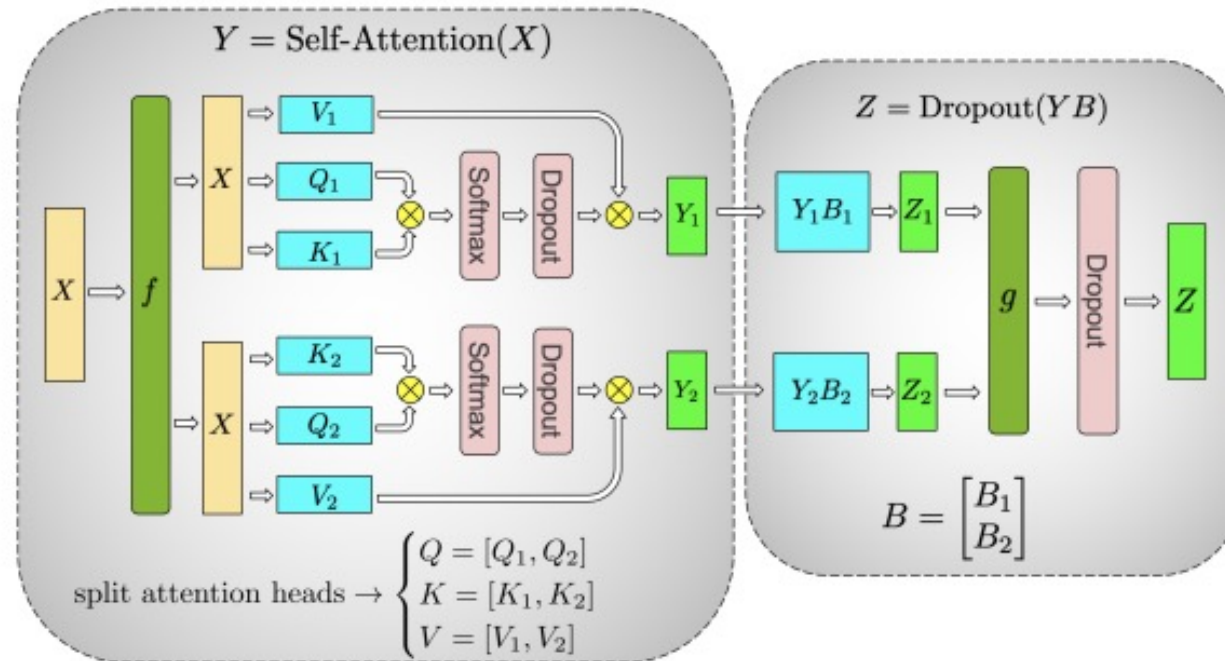
Pipeline Parallelism

- P2P communication volume:
 - Assume the computation and the communication are all in FP16.
 - L is the input sequence length;
 - D is the model dimension.
 - Prefill stage: $2LD$ bytes.
 - Decode stage: $2D$ bytes for each generated token.

Tensor Model Parallelism

- Tensor model parallelism partitions the inference computation at the level of transformer layers over multiple GPUs, where the weight matrices are distributed both row-wisely and column-wisely.
- Two **AllReduce** operations are required to aggregate each layer's output activations:
 - One **AllReduce** for the Multihead-Attention.
 - One **AllReduce** for the MLP.
- Tensor model parallelism splits both the data scan and computation among a tensor model parallel group, which can effectively scale out the inference computation if the connection is fast among the group.

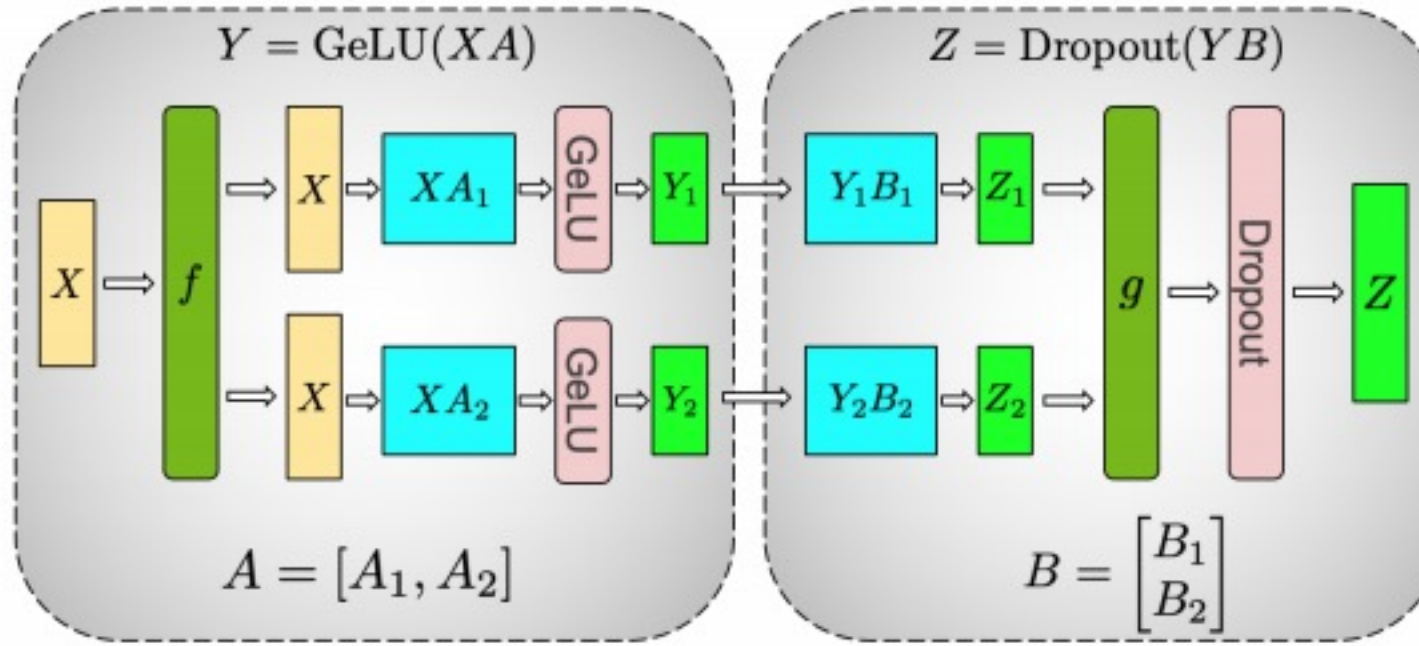
Multi-Head Attention in Tensor Model Parallelism



(b) Self-Attention

- f is the identity operator in the forward pass and the **AllReduce** operator in the backward pass.
- g is the **AllReduce** operator in the forward pass and the identity operator in the backward pass.

MLP in Tensor Model Parallelism



(a) MLP

- f is the identity operator in the forward pass and the ~~AllReduce~~ operator in the backward pass.
- g is the ~~AllReduce~~ operator in the forward pass and the identity operator in the backward pass.

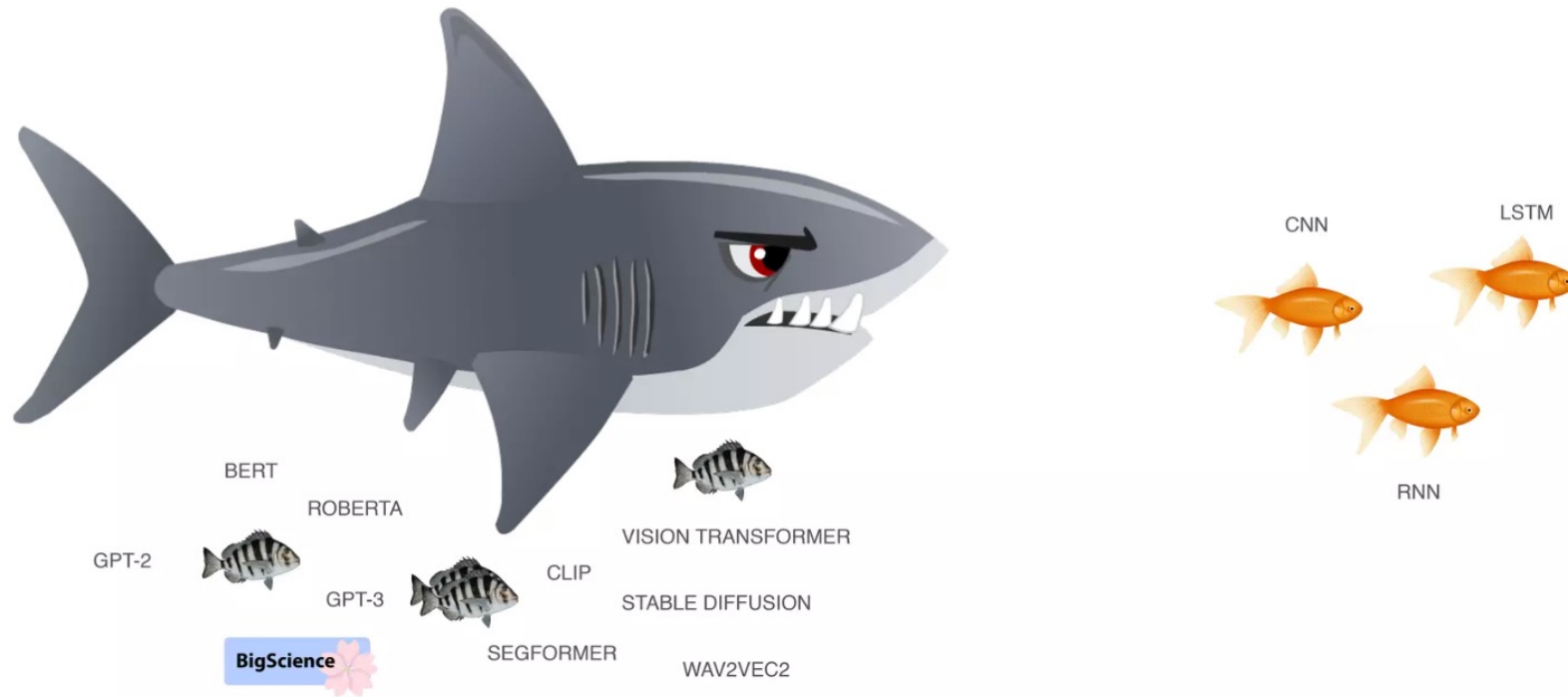
Tensor Model Parallelism

- Collective communication volume:
 - Assume the computation and the communication are all in FP16.
 - L is the input sequence length;
 - D is the model dimension.
 - Prefill stage:
 - For each layer, two **AllReduces**, where each aggregates $2LD$ bytes.
 - Decode stage:
 - For each generated token, each layer, two **AllReduces**, where each aggregates $2D$ bytes.



Hugging Face

Transformers Are Eating Deep Learning



"Transformers are emerging as a general-purpose architecture for ML"

<https://www.stateof.ai/>

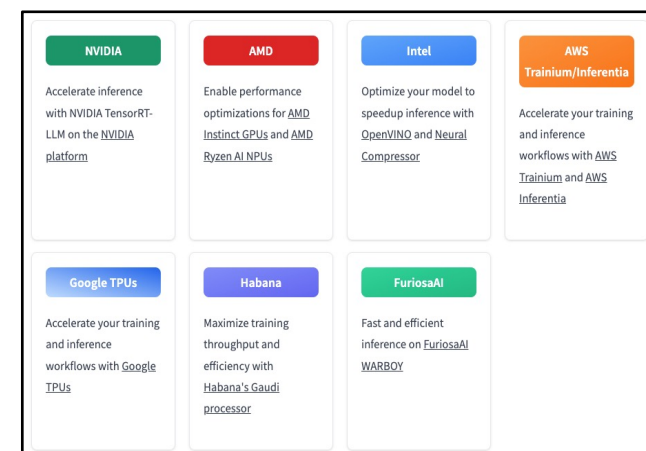
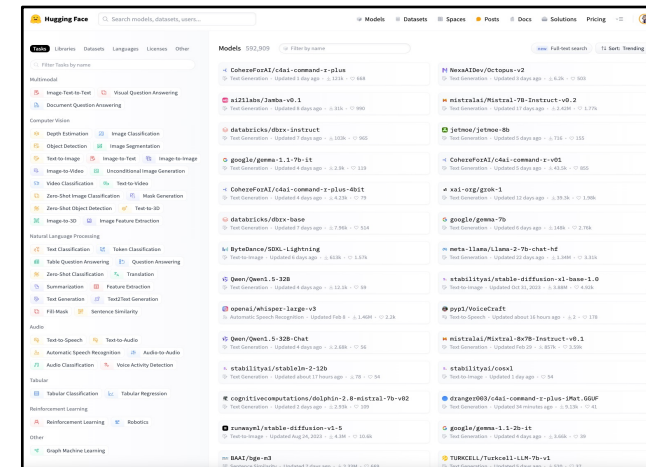
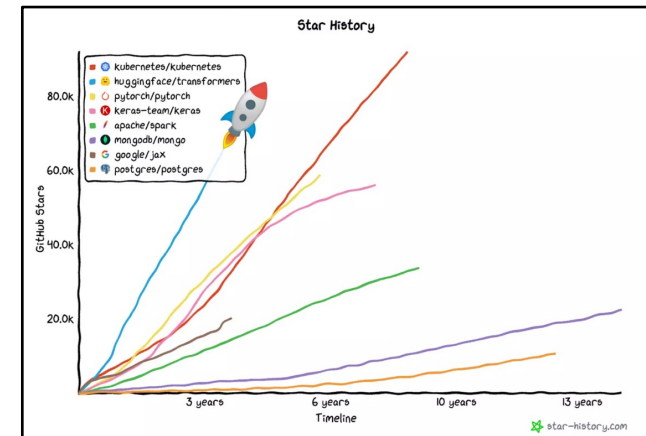
RNN and CNN usage down, Transformers usage up!

<https://www.kaggle.com/kaggle-survey-2021>

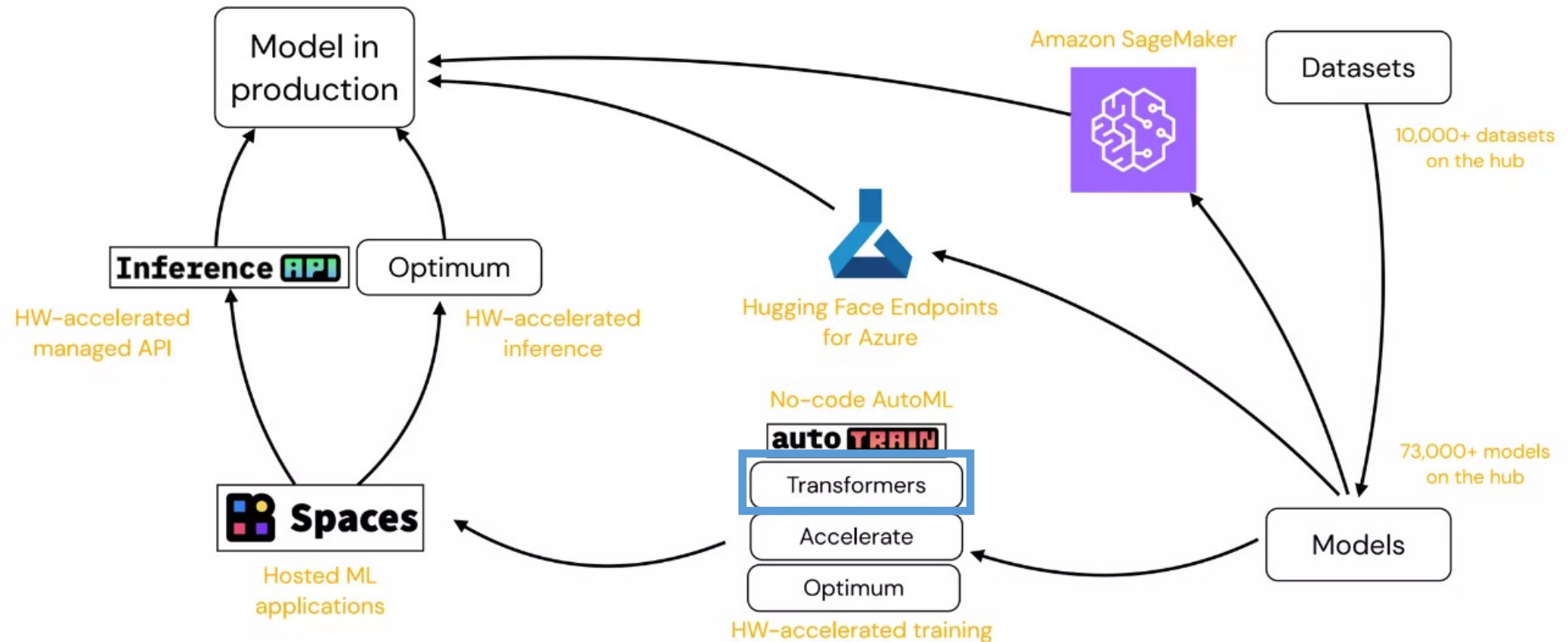


Hugging Face

- Hugging Face Transformers:
 - Provide APIs and tools to easily download and train state-of-the-art pre-trained models.
- Hugging Face Hub:
 - The Github of machine learning models.
- Hugging Face Optimum:
 - Provides a set of performance optimization tools for deployment on various hardware.



Hugging Face Ecosystem



Hugging Face Transformers

Transformers Overview

- Hugging Face Transformers provides APIs and tools to download and train state-of-the-art pre-trained models easily.
- Transformers support frameworks of PyTorch, TensorFlow, and JAX for the implementation of those pre-trained models.
- Users can use a different framework at each stage of a model's life:
 - Train a model in three lines of code in one framework;
 - Load it for inference in another;
 - Models can also be exported to a format like ONNX and TorchScript for deployment in production environments.

Pipelines

- The pipelines are the easiest way to use models for inference.
- These pipelines are objects that abstract most of the complex code from the library, offering a simple API dedicated to a list of tasks.
- Two categories of pipeline abstractions:
 - The **pipeline()** which is the most powerful object encapsulating all other pipelines.
 - *Task-specific pipelines* are available for audio, computer vision, natural language processing, and multimodal tasks.



Audio
Audio classification
Automatic speech recognition
Computer vision
Image classification
Object detection
Image segmentation
Depth estimation
Natural language processing
Text classification
Token classification
Question answering
Summarization
Translation
Language modeling
Multimodal
Document question answering

List of Tasks in Pipelines

Task	Description	Modality	Pipeline identifier
Text classification	assign a label to a given sequence of text	NLP	pipeline(task="sentiment-analysis")
Text generation	generate text given a prompt	NLP	pipeline(task="text-generation")
Summarization	generate a summary of a sequence of text or document	NLP	pipeline(task="summarization")
Image classification	assign a label to an image	Computer vision	pipeline(task="image-classification")
Image segmentation	assign a label to each individual pixel of an image (supports semantic, panoptic, and instance segmentation)	Computer vision	pipeline(task="image-segmentation")
Object detection	predict the bounding boxes and classes of objects in an image	Computer vision	pipeline(task="object-detection")
Audio classification	assign a label to some audio data	Audio	pipeline(task="audio-classification")
Automatic speech recognition	transcribe speech into text	Audio	pipeline(task="automatic-speech-recognition")
Visual question answering	answer a question about the image, given an image and a question	Multimodal	pipeline(task="vqa")
Document question answering	answer a question about the document, given a document and a question	Multimodal	pipeline(task="document-question-answering")
Image captioning	generate a caption for a given image	Multimodal	pipeline(task="image-to-text")

Pipelines API

 transformers.pipeline 

<source>

```
( task: str = None, model: Union = None, config: Union = None, tokenizer: Union = None,
feature_extractor: Union = None, image_processor: Union = None, framework: Optional = None, revision:
Optional = None, use_fast: bool = True, token: Union = None, device: Union = None, device_map = None,
torch_dtype = None, trust_remote_code: Optional = None, model_kwargs: Dict = None, pipeline_class:
Optional = None, **kwargs ) → Pipeline
```

https://huggingface.co/docs/transformers/v4.39.3/en/main_classes/pipelines#transformers.pipeline

- Pipelines are made of:
 - A **tokenizer** in charge of mapping raw textual input to token.
 - A **model** to make predictions from the inputs.
 - **Some (optional) post processing** for enhancing model's output.

Pipeline Text Classification Example

- Text Classification Example:

Code	Output
<pre>from transformers import pipeline pipe = pipeline(task="text-classification", model="distilbert/distilbert-base-uncased-finetuned-sst-2-english") result1 = pipe("The food in HKUST is terrible") print(result1) result2 = pipe("This course is awesome") print(result2)</pre>	<pre>[{'label': 'NEGATIVE', 'score': 0.9986492991447449}] [{'label': 'POSITIVE', 'score': 0.9998718500137329}]</pre>

Pipeline Translation Example

- Translation Example:

Code	Output
<pre>from transformers import pipeline text = "translate English to French: Hugging Face is a community-based open-source platform for machine learning." translator = pipeline(task="translation", model="google/flan-t5-small") text_in_chinese = translator(text) print(text_in_chinese)</pre>	<pre>[{'translation_text': "Hugging Face est un platform d'open-source pour l'apprentissage de machine."}]</pre>

Pipeline Summarization Example

- Summarization Example:

Code	Output
<pre>from transformers import pipeline summarizer = pipeline(task="summarization", model="google/flan-t5-small", max_new_tokens=9) text = "Summarization creates a shorter version of a text from a longer one while trying to preserve most of the meaning of the original document. " result = summarizer(text) print(result)</pre>	<pre>[{'summary_text': 'Summarization creates a shorter version'}]</pre>

Pipeline Language Modeling Example

- Language modeling Example:

Code	Output
<pre>from transformers import pipeline generator = pipeline(task="text-generation", model="facebook/opt-350m") prompt_text = "Which university is best in Europe?" result = generator(prompt_text) print(result)</pre>	<pre>[{'generated_text': 'Which university is best in Europe?\n\nThe University of Oxford is the best university in Europe.'}]</pre>

Model API

- **Model** is the base class implement the common methods for loading/saving a model:
 - From a local file or directory or;
 - From a pretrained model configuration provided by the library (downloaded from HuggingFace's AWS S3 repository).
- Model class implements a methods which are common among all the models to, e.g.:
 - Resize the input token embeddings when new tokens are added to the vocabulary;
 - Prune the attention heads of the model.
- PyTorch: **PreTrainedModel**;
- TensorFlow: **TFPreTrainedModel**;
- Flax: **FlaxPreTrainedModel**.

Model API



RELAXED
SYSTEM LAB

`class transformers.PreTrainedModel` [↗](#) < source >

```
( config: PretrainedConfig, *inputs, **kwargs )
```

```
( repo_id: str, use_temp_dir: Optional = None, commit_message: Optional = None, private: Optional =  
None, token: Union = None, max_shard_size: Union = '5GB', create_pr: bool = False, safe_serialization:  
bool = True, revision: str = None, commit_description: str = None, tags: Optional = None,  
**deprecated_kwargs )
```

Parameters

- **repo_id** (str) — The name of the repository you want to push your model to. It should contain your organization name when pushing to a given organization.
- **use_temp_dir** (bool, *optional*) — Whether or not to use a temporary directory to store the files saved before they are pushed to the Hub. Will default to True if there is no directory named like repo_id, False otherwise.
- **commit_message** (str, *optional*) — Message to commit while pushing. Will default to "Upload model".
- **private** (bool, *optional*) — Whether or not the repository created should be private.
- **token** (bool or str, *optional*) — The token to use as HTTP bearer authorization for remote files. If True, will use the token generated when running `huggingface-cli login` (stored in `~/.huggingface`). Will default to True if `repo_url` is not specified.
- **max_shard_size** (int or str, *optional*, defaults to "5GB") — Only applicable for models. The maximum size for a checkpoint before being sharded. Checkpoints shard will then be each of size lower than this size. If expressed as a string, needs to be digits followed by a unit (like "5MB"). We default it to "5GB" so that users can easily load models on free-tier Google Colab instances without any CPU OOM issues.
- **create_pr** (bool, *optional*, defaults to False) — Whether or not to create a PR with the uploaded files or directly commit.
- **safe_serialization** (bool, *optional*, defaults to True) — Whether or not to convert the model weights in safetensors format for safer serialization.
- **revision** (str, *optional*) — Branch to push the uploaded files to.
- **commit_description** (str, *optional*) — The description of the commit that will be created
- **tags** (List[str], *optional*) — List of tags to push on the Hub.

Tokenizer API

- A tokenizer is in charge of preparing the inputs for a model. The library contains tokenizers for all the models.
- The base class e.g., **PreTrainedTokenizer** implement the common methods for encoding string inputs as model inputs and instantiating/saving tokenizers either from a local file or directory or from a pretrained tokenizer provided by the library (downloaded from HuggingFace's AWS S3 repository).
- Tokenizer implements the main methods for using all the tokenizers:
 - Tokenizing (splitting strings in sub-word token strings), converting tokens strings to ids and back, and encoding/decoding (i.e., tokenizing and converting to integers).
 - Adding new tokens to the vocabulary in a way that is independent of the underlying structure (BPE, SentencePiece...).
 - Managing special tokens (like mask, beginning-of-sentence, etc.): adding them, assigning them to attributes in the tokenizer for easy access and making sure they are not split during tokenization.

Tokenizer API

`class transformers.PreTrainedTokenizer` [🔗](#)

[<source>](#)

`( **kwargs )`

Class attributes (overridden by derived classes)

- **vocab_files_names** (`Dict[str, str]`) — A dictionary with, as keys, the `__init__` keyword name of each vocabulary file required by the model, and as associated values, the filename for saving the associated file (string).
- **pretrained_vocab_files_map** (`Dict[str, Dict[str, str]]`) — A dictionary of dictionaries, with the high-level keys being the `__init__` keyword name of each vocabulary file required by the model, the low-level being the short-cut-names of the pretrained models with, as associated values, the url to the associated pretrained vocabulary file.
- **max_model_input_sizes** (`Dict[str, Optional[int]]`) — A dictionary with, as keys, the short-cut-names of the pretrained models, and as associated values, the maximum length of the sequence inputs of this model, or `None` if the model has no maximum input size.
- **pretrained_init_configuration** (`Dict[str, Dict[str, Any]]`) — A dictionary with, as keys, the short-cut-names of the pretrained models, and as associated values, a dictionary of specific arguments to pass to the `__init__` method of the tokenizer class for this pretrained model when loading the tokenizer with the `from_pretrained()` method.
- **model_input_names** (`List[str]`) — A list of inputs expected in the forward pass of the model.
- **padding_side** (`str`) — The default value for the side on which the model should have padding applied. Should be `'right'` or `'left'`.
- **truncation_side** (`str`) — The default value for the side on which the model should have truncation applied. Should be `'right'` or `'left'`.

Generation API

- Each framework has a generate method for text generation implemented in their respective **GenerationMixin** class:
 - PyTorch **generate()** is implemented in **GenerationMixin**.
 - TensorFlow **generate()** is implemented in **TFGenerationMixin**.
 - Flax/JAX **generate()** is implemented in **FlaxGenerationMixin**.
- Regardless of the framework of choice, one can parameterize the generate method with a **GenerationConfig** class instance.

Customize Text Generation

https://huggingface.co/docs/transformers/generation_strategies#customize-text-generation

- Some of the commonly adjusted parameters include:
 - **max_new_tokens**: the maximum number of tokens to generate. In other words, the size of the output sequence, not including the tokens in the prompt. Or you can implement some **StoppingCriteria**.
 - **num_beams**: by specifying a number of beams higher than 1, you are effectively switching from greedy search to beam search. This strategy evaluates several hypotheses at each time step and eventually chooses the hypothesis that has the overall highest probability for the entire sequence. This has the advantage of identifying high-probability sequences that start with a lower probability initial tokens and would've been ignored by the greedy search.
 - **do_sample**: if set to True, this parameter enables decoding strategies such as multinomial sampling, beam-search multinomial sampling, Top-K sampling and Top-p sampling. All these strategies select the next token from the probability distribution over the entire vocabulary with various strategy-specific adjustments.
 - **num_return_sequences**: the number of sequence candidates to return for each input. This option is only available for the decoding strategies that support multiple sequence candidates, e.g. variations of beam search and sampling. Decoding strategies like greedy search and contrastive search return a single output sequence.

Customize Text Generation: Beam Search

- A variant of breadth-first search (BFS) to expand the search space of greedy search.
 - Greedy search: in every generation step, keep the token with the highest probability.
- Beam search: keep top-k (**num_beams**=k) tokens for generation.
 - For the first decode step, the k words with the highest current target value (e.g., conditional probability) are selected as the first word of the candidate output sequence.
 - For each subsequent decode step, based on the output sequence of the previous step, select k largest target value (e.g., conditional probability) in all combinations as the candidate output sequence by the current step.
 - It always keeps k candidates and picks the best of the candidates at the end.

Greedy Search Example

- Three tokens: {I, L, U}

	Step 1	Step 2	Step 3
I	0.6	0.2	0.05
L	0.3	0.7	0.05
U	0.1	0.1	0.9

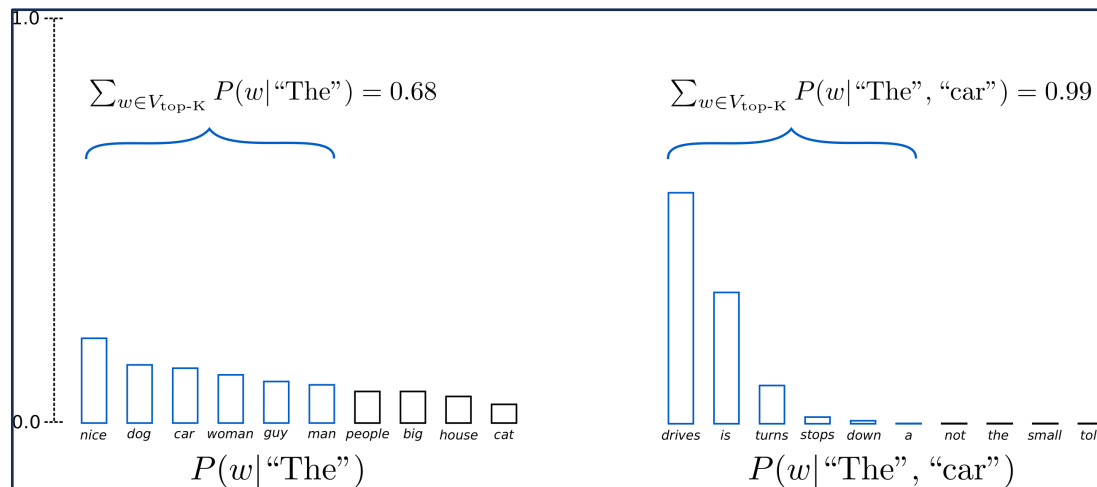
Beam Search Example

- Three tokens: {I, L, U}
- $k = 2$

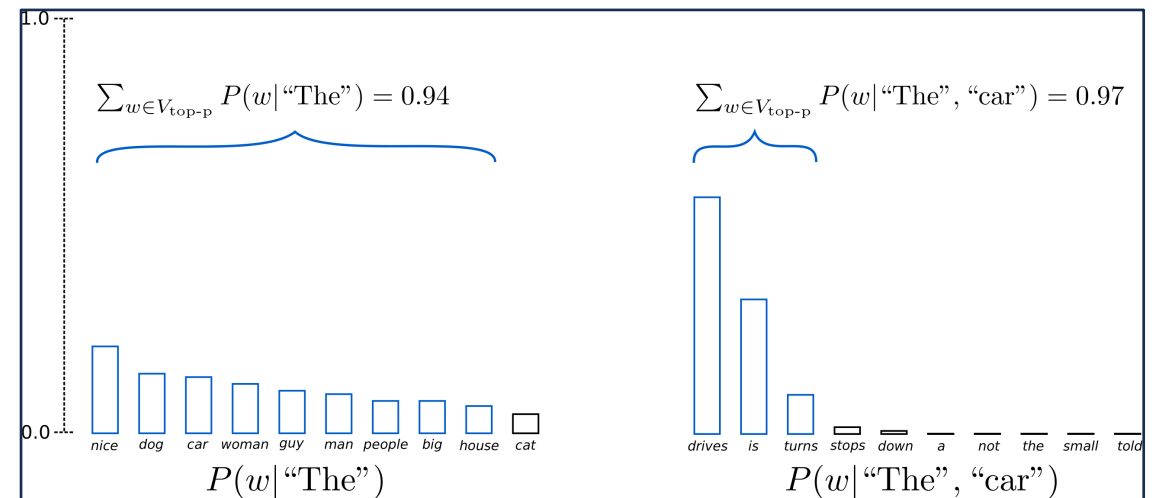
Step 1			Step 2-1 (I)			Step 3-1 (IL)			Step 4-1 (ILL)			Step 4-1 (ILU)		
I	0.6		I	0.2	0.12	I	0.1	0.042						
L	0.3		L	0.7	0.42	L	0.2	0.084						
U	0.1		U	0.1	0.06	U	0.7	0.294						
			Step 2-2 (L)			Step 3-2 (LI)								
			I	0.5	0.15	I	0.3	0.045						
			L	0.2	0.06	L	0.4	0.06						
			U	0.3	0.09	U	0.3	0.045						

Sampling

- **Sampling**: instead of deterministic selecting the largest tokens, we use a random number generator to sample tokens following the distribution computed by the LM.
- **Top-k sampling**: only the k (e.g., $k = 6$) most likely next words are filtered to be sampled.
- **Top-p sampling**: chooses from the smallest possible set of words whose cumulative probability exceeds the probability p (e.g., $p = 0.92$).



Top-k sampling



Top-p sampling

An Inference Example

Define the Generate Function

Code

```
import argparse
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers.models.opt.modeling_opt import *

def generate(task_info, device, model, tokenizer):
    contexts = task_info["prompt_seqs"]
    inputs = tokenizer(contexts, return_tensors="pt").to(device)
    print(f"start_ids: length ({inputs.input_ids.shape[0]}) ids: {inputs.input_ids}")
    input_length = inputs.input_ids.shape[1]

    outputs = model.generate(
        *inputs, do_sample=True, top_p=task_info['top_p'],
        temperature=1.0, top_k=1,
        max_new_tokens=task_info["output_len"],
        return_dict_in_generate=True,
        output_scores=True, # return logit score
        output_hidden_states=False, # return embeddings
    )
    print(f"[INFO] raw output: {outputs.keys()} {len(outputs)}, {outputs[0].shape},
    ({outputs[1][0].shape},{outputs[1][1].shape}) {len(outputs[2])}")
    token = outputs.sequences[0, input_length:] # exclude context input from the output
    print(f"[INFO] raw token: {token}")
    output = tokenizer.decode(token)
    print(f"[INFO] \n[Context]\n{contexts}\n\n[Output]\n{output}\n")
```


Test the Model

Code

```
def test_model(args):
    print(f"<test_model> initialization start")
    device = torch.device(args.get('device', 'cpu'))
    tokenizer = AutoTokenizer.from_pretrained(args['hf_model_name'])
    model = AutoModelForCausalLM.from_pretrained(args['hf_model_name'])
    model = model.to(device)
    torch.manual_seed(0)
    task_info = {
        "seed": 0,
        "prompt_seqs": None,
        "output_len": 16,
        "beam_width": 1,
        "top_k": 50,
        "top_p": 1,
        "beam_search_diversity_rate": 0,
        "len_penalty": 0,
        "repetition_penalty": 1.0,
        "stop": args.get("stop", []),
        "logprobs": 5,
    }
    print(f"<test_model> initialization done")

    if args["interactive"]:
        while True:
            prompt_data = input("Please enter the prompt input:\n")
            task_info["prompt_seqs"] = prompt_data
            generate(task_info, device, model, tokenizer)
    else:
        generate(task_info, device, model, tokenizer)
```

The Main Entrance

Code

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument('--hf_model_name', type=str, default='facebook/opt-350m',
                        help='hugging face model name (used to load config).')
    args = parser.parse_args()
    test_model(args={
        "hf_model_name": args.hf_model_name,
        "interactive": True,
        "device": "cpu",
        "dtype": torch.float32,
    })
```

References

- <https://arxiv.org/abs/2402.16363>
- https://huggingface.co/docs/transformers/en/llm_tutorial
- <https://www.youtube.com/watch?v=fckyXntHy1s&t=1070s>
- <https://huggingface.co/docs/transformers/index>
- https://huggingface.co/docs/transformers/generation_strategies
- <https://huggingface.co/blog/how-to-generate>